KALSHI BOT

Complete Research & Build Reference

For Claude Code — Autonomous Trading Bot Development

Generated March 2026 | Matthew's POLYBOT Project | Post-Grand Rounds Build Target

SECTION 1: PURPOSE & CLAUDE CODE INSTRUCTIONS

This document is the complete, unabridged reference for the Kalshi trading bot project. It contains every research finding, every question asked, every argument for and against, every code specification, every tax consideration, every plugin instruction, and every edge-detection methodology discussed across the full research conversation. Nothing has been summarized or omitted. Claude Code should read this document at the start of every session and treat it as the authoritative project context.

CLAUDE CODE INSTRUCTION:
At session start, read this entire document. Treat it as CLAUDE.md-level context. Every decision made in this project must be consistent with the constraints, risk limits, and architecture described here. Do not skip sections.

Hard Constraints — Never Violate These

- DRY_RUN=true until 200+ paper trades logged with positive net EV after fees
- $5 maximum per trade — enforced programmatically, not just by convention
- $25 daily loss kill switch — auto-halt, reset at midnight UTC
- Limit orders ONLY — no market orders under any circumstances
- All trades logged to SQLite with: timestamp, market_ticker, side, contracts, entry_price, exit_price, gross_profit, kalshi_fee, net_profit, tax_basis, dry_run
- Brier score must drop below 0.20 across 100+ resolved markets before live capital
- Kelly Criterion position sizing — never gut-feel sizing
- Kill switch cancels all open orders and halts bot on: daily loss limit breach, API error rate >10% in 5 min, price feed failure >30 seconds
- This is a post-Grand Rounds (~March 20, 2026) build — do not start Phase 1 before that date

---

SECTION 2: KALSHI MARKET RESEARCH — COMPLETE FINDINGS

2.1 Is Kalshi Bot Trading Actually Profitable?

The objective answer based on academic research and practitioner data: Yes, but the edge categories are narrow, rapidly compressing, and require specific infrastructure to capture.

The average pre-fee rate of return on Kalshi contracts is minus 20%. Even before fees, contracts are losing propositions on average. This is the fundamental baseline. Fees then layer on top of that negative expectation. Any bot strategy that is not explicitly maker-heavy or arbitrage-based is fighting a 20%+ headwind before a single edge-based trade.

Taker fees run 0.07% to 7% depending on contract probability, peaking near 50/50 odds. Maker fees are $0 on most markets. At high turnover, the compounding is brutal: a trader with $10,000 and 10x annual turnover needs a 12-point edge just to cover fees before showing any return. For a bot trading BTC 15-minute markets, turnover could be 50x+.

2.2 The Structural Biases That Create Exploitable Edges

Academic research (GWU/UCD working papers on the full Kalshi dataset) found:

- Favorite-longshot bias is statistically significant and persistent across all contract categories
- Contracts priced below 10 cents lose over 60% of invested capital
- Contracts above 50 cents earn a small positive rate of return
- The bias appears across all market categories, though weakening in 2025 data
- Takers show more pronounced favorite-longshot bias than makers — makers set more rational prices, takers are overoptimistic about longshots

Translation: The two exploitable structural edges are (1) fading longshots (systematic NO side on <10-cent contracts) and (2) maker liquidity provision on high-probability contracts (>50 cents).

2.3 The Three Real Edge Categories for Bots

Edge 1: Single-Platform Probability Arbitrage (YES + NO ≠ $1.00)

When YES + NO sum falls below $1.00, buying both guarantees profit at resolution regardless of outcome. The gap must be large enough to survive fees, slippage, and execution time. The average arbitrage opportunity duration has compressed from 12.3 seconds in 2024 to 2.7 seconds as of Q1 2026. 73% of arbitrage profits are now captured by sub-100ms execution bots. The median arbitrage spread is now ~0.3% — barely profitable after fees. This edge is largely closed for retail.

Edge 2: Cross-Platform Arbitrage (Kalshi vs. Polymarket)

Between April 2024 and April 2025, traders extracted an estimated $40 million in cross-platform arbitrage profits, with bots driving the majority. Targets need spreads of 2.5-3% minimum to offset fees and ensure gains. Most conditions offer only $50-$500 maximum profit at available liquidity. Sub-second execution required for high-frequency opportunities.

Edge 3: Market Making / Liquidity Provision

Since Kalshi maker fees are $0 on most markets, this is the lowest-friction edge available. Market makers on comparable platforms have reported $200-$800/day during high-volume periods. Win rates of 78-85% with 1-3% monthly returns are cited for systematic market-making bots. Market making is the edge nobody talks about because it is not exciting. There are no viral screenshots. Just slow, steady, compound growth.

2.4 The BTC Price Lag Strategy — Your Specific Target

The core mechanism: If options markets imply a 62% probability that Bitcoin will close above a certain level over a short time window, but a Kalshi prediction market contract tied to the same outcome suggests only a 55% probability, a discrepancy emerges. Automated traders can monitor both venues simultaneously, compare implied probabilities, and buy whichever side appears mispriced.

The documented real-world result: A bot designated 0x8dxd turned $313 into approximately $437,600 in one month on Polymarket in December 2025, using a strategy of monitoring BTC spot prices on Binance and Coinbase and trading when the platform had not yet accounted for the growing price certainty. That result is an extraordinary outlier and survivorship bias is rampant in public reports.

Critical counterpoint: Bocconi quantitative research on this exact approach — backtesting a GBM-based pricing model against Kalshi BTC contracts — concluded the weekly Kalshi contracts had abhorrently high bid-ask spreads on the orderbook, with frequent instances where most strikes had a 1-100 bid-ask spread, making divergences non-tradeable even when they existed. They pivoted to Polymarket hourly data because Kalshi lacked the liquidity to execute. The BTC 15-minute market on Kalshi has different (shorter) interval mechanics and may have better liquidity than the weekly contracts studied, but this remains unvalidated in peer-reviewed literature.

A fully automated trading bot executed 8,894 trades on short-term crypto prediction contracts and reportedly generated nearly $150,000 without human intervention, exploiting moments when YES+NO contracts summed to less than $1, locking in roughly 1.5-3% per trade. Typical five-minute crypto prediction markets showed only $5,000-$15,000 per side in depth, meaning large desks would struggle to deploy serious capital without erasing the spread.

2.5 Liquidity Scalability — The Hard Ceiling

Kalshi's trading activity shows extreme volume concentration: just 7 categories account for 50% of total volume and most markets remain thinly traded. This creates a challenging problem where users avoid low-liquidity markets, which prevents those markets from developing sufficient liquidity. A 2025 study found 78% of arbitrage opportunities in low-volume markets failed due to execution inefficiencies. Combined fees of 5%+ mean spreads under 5% are fee-negative.

Capital efficiency is the core constraint: most conditions offer only $50-$500 maximum profit at available liquidity. For a retail operator, this means the realistic ceiling before liquidity constraints kick in is modest — probably in the low hundreds of dollars per week on a focused strategy with limited capital. You are not replacing income. You are testing whether a strategy can generate $50-$200/week net of fees with a $500-$2,000 bankroll while learning the infrastructure. That is the honest framing.

2.6 Hard Requirements for Sustainable Profitability

Requirement	Threshold / Specification
Execution speed	Sub-100ms for arb; limit orders for making
Minimum spread target	>2.5-3% for cross-platform arb
Model calibration	Brier score verified over 100+ resolved markets
Capital (starting)	$200-$500 practical floor; <$200 is noise
Capital (scaling)	$500-$2,000 bankroll once strategy validated
Infrastructure	Low-latency VPS (Chicago proximity for Kalshi)
API tier	Premier tier for 100 req/s vs Basic 10 req/s
Market focus	High-volume categories only: sports, BTC, macro
Position sizing	Kelly Criterion — never gut-feel
Fee calculation	Explicit per-trade: ceil(0.07 x contracts x price x (1-price))

2.7 Platform Risk Factors

- Kalshi halted a major geopolitical market and settled at last traded price rather than $1, triggering trader backlash and acknowledging 'grammatically ambiguous' settlement language
- Trustpilot reviews cite withdrawal failures, ID verification delays lasting weeks, and payouts that do not calculate correctly
- Kalshi is explicitly banned in at least four US states at the state level despite federal approval
- Massachusetts sued Kalshi in September 2025 for allegedly operating unlicensed sports betting with over $1 billion in wagers during H1 2025
- Rate limit breach bans your bot — Basic tier: 20 read requests/second, 10 write requests/second; Premier tier: up to 100 requests/second
- One practitioner who built and deployed a Kalshi sports bot won over 66% of trades and still lost money because fees consumed the edge on sports markets

SECTION 3: IS KALSHI THE RIGHT PLATFORM?

3.1 Kalshi vs. Polymarket — Decision Framework

For American traders, Kalshi currently maintains the edge in instant accessibility. Polymarket US is now CFTC-regulated too following its $112 million acquisition of QCEX in July 2025, but as of early 2026, access remains restricted through an invite-only waitlist. Hundreds of thousands of users are still waiting.

On major markets like elections and crypto, Polymarket has 10-50x more liquidity than Kalshi. Lower liquidity equals wider bid-ask equals harder to fill at your edge price. For most trades priced between 30 cents and 70 cents, Polymarket's fee model (0.10% flat taker) is often cheaper than Kalshi's probability-weighted formula.

Kalshi lists over 350,000 active markets across sports, politics, economics, and crypto. Polymarket US launched with sports only and remains gated behind a waitlist. Polymarket has historically had 10-50x more liquidity on major markets.

3.2 Current Answer: Kalshi for Now, Build Platform-Agnostic Code

For a bot targeting BTC short-interval markets specifically, Kalshi is correct right now because Polymarket US is still invite-only. But this calculus changes as Polymarket rolls out. Build platform-agnostic code from day one. Your kalshi_client.py should be swappable with a polymarket_client.py without rewriting edge_detector.py, risk_manager.py, or logger.py.

The BTC short-interval markets run continuously on both platforms, and cross-platform arbitrage between them is a documented strategy — though the Rust-based bots with co-located VPS are capturing most of those opportunities in under 2.7 seconds.

3.3 Fee Formula — Applied to Every Trade

Kalshi taker fee formula: fee = ceil(0.07 x contracts x price x (1 - price))

At price = 0.50 (worst case, 50/50 market): fee = ceil(0.07 x 1 x 0.5 x 0.5) = ceil(0.0175) = 2 cents per contract

At price = 0.85 (high-probability contract): fee = ceil(0.07 x 1 x 0.85 x 0.15) = ceil(0.0089) = 1 cent per contract

At price = 0.10 (longshot contract): fee = ceil(0.07 x 1 x 0.10 x 0.90) = ceil(0.0063) = 1 cent per contract

BTC markets late in their window often price near extremes (high probability). Fees appear low on a per-contract basis but must be modeled explicitly at the trade level against edge size. The fee simulator in your bot is non-negotiable — you cannot assume edge survives fees without explicit calculation.

SECTION 4: TAX TREATMENT — COMPLETE ANALYSIS

4.1 The Three Possible Tax Treatments

There is currently no IRS-specific guidance on prediction market contracts. Tax advisors agree income must be reported but are divided on classification. Three possible treatments exist:

Treatment A: Ordinary Income (Most Conservative)

Report as 'Other Income' on Schedule 1, Line 8z with description 'Prediction Market Earnings.' Taxed at your full ordinary income rate (22-32% for a PGY-3 resident depending on total income including moonlighting). In practice, most conservative approaches treat Kalshi contracts as non-equity, cash-settled instruments and report net results as ordinary income. This approach prioritizes consistency, documentation, and audit defensibility in the absence of clear statutory direction.

Treatment B: Section 1256 Contracts (Most Favorable — Also Most Contested)

Section 1256 provides a 60/40 split: 60% of net gain treated as long-term capital gain, 40% as short-term, regardless of actual holding period. Reported on Form 6781. Because Kalshi has always operated as a CFTC-designated contract market, some practitioners argue its contracts have unambiguously qualified as Section 1256 contracts. Kalshi users under this treatment would also be shielded from the new 90% gambling loss cap under

the One Big Beautiful Bill Act because losses would be classified as capital losses (which can be carried back three years) rather than itemized gambling deductions.

The counterargument: Section 1256 treatment is unsettled for event-based prediction contracts and should not be viewed as a default or generally advisable position. Regulatory status alone does not convert an instrument into a Section 1256 contract if the statutory requirements are not otherwise satisfied. At present, there is no IRS guidance directly addressing prediction market contracts under Section 1256.

Treatment C: Gambling Income

Ordinary income rates but with deduction limitations. Under the One Big Beautiful Bill Act, gambling losses are now subject to a 90% cap starting January 1, 2026 (your 2025 return still uses the old 100% loss deduction rule). Some states classify prediction market income as gambling income and require separate reporting.

4.2 Practical Recommendation

For a PGY-3 resident without a dedicated CPA specializing in prediction markets: use ordinary income reporting (Treatment A) initially. It is the most defensible and requires the least documentation burden. If annual profits exceed $5,000, engage Camuso CPA or Monaco CPA (both specialize in prediction market taxes) to evaluate whether a Section 1256 position is supportable and worth the analytical burden.

4.3 Tax Math Applied to the $50/Day Target

Metric	Gross	Kalshi Fees	Pre-tax Net	Tax at 24% (ordinary)	After-tax retained
Per day (target)	$50.00	Variable (~$3-8)	~$42-47	~$10-11	~$31-37
Per week (3 active days)	$150.00	~$12-24	~$126-138	~$30-33	~$96-105
Per year (150 active days)	$7,500	~$600-1,200	~$6,300-6,900	~$1,512-1,656	~$4,644-5,388

Quarterly estimated taxes are mandatory if annual profit exceeds approximately $1,000. Prediction market earnings are not subject to W-2 withholding, so you must make quarterly payments to avoid underpayment penalties.

Kalshi uses FIFO accounting and their PnL figure includes fees and rebates already netted. Do not trust Kalshi's summary reporting for tax reconstruction — their platform reports are informational summaries, not determinations of correct tax characterization. Your bot must independently log every trade to SQLite with full detail.

4.4 Required Tax Logging Fields (Every Trade)

- id (auto-increment primary key)
- timestamp (ISO 8601, UTC)
- market_ticker (Kalshi market ID)
- side (YES or NO)
- contracts (integer count)
- entry_price (cents, 0-99)
- exit_price (cents at resolution or early exit)
- gross_profit (dollars, before fees)
- kalshi_fee (dollars, actual fee charged — use ceil(0.07 x contracts x price x (1-price)))
- net_profit (gross_profit minus kalshi_fee)
- tax_basis (= net_profit, for ordinary income reporting)
- brier_input (probability forecast made at entry)
- outcome (1 if correct, 0 if wrong — populated at resolution)
- dry_run (boolean)

SECTION 5: THE $50/DAY GOAL — OBJECTIVE ASSESSMENT

5.1 Is $50/Day Achievable?

The honest, math-grounded answer: Achievable in theory, unproven in practice for your specific strategy on Kalshi, and dependent on edge validation that has not yet occurred.

The mechanism that makes it mathematically possible: A fully automated trading bot executed 8,894 trades on short-term crypto prediction contracts and reportedly generated nearly $150,000 without human intervention, exploiting moments when YES+NO contracts briefly summed to less than $1, locking in roughly 1.5-3% per trade. That works out to roughly $16.80 in profit per trade. To gross $50/day, you need approximately 3 trades at that rate with adequate liquidity. On BTC 15-minute markets running 24/7 with 96 intervals per day, 3 qualifying trades per day is plausible if the edge exists.

The mechanism that makes it hard: The median arbitrage spread has compressed to 0.3% — barely profitable after fees. The BTC lag strategy requires your spot price data feed to be faster than Kalshi's internal pricing update. That is a speed/infrastructure problem, not a prediction problem. A practical documented case: a developer earned $764 in a single day on December 21, 2025 using an automated BTC-15m bot with only a $200 initial deposit. That is real, but it is a single data point.

5.2 The Cautionary Counter-Data

One practitioner who built and deployed a Kalshi sports bot won over 66% of trades and still lost money. He identified two reasons: fees consumed the edge, and the strategy lacked a validated model — it was directional intuition, not calibrated probability estimation. The lesson applies directly: win rate alone is meaningless without calibrated edge that survives fees.

Less than 1% of the day trader population is able to predictably and reliably earn positive abnormal returns net of fees. A disproportionate amount of profits go to a small number of participants. Do not expect to make money without the Brier-gate validated infrastructure described in this document.

5.3 Verdict

PURSUE IF AND ONLY IF:
Backtest on historical BTC 15m market data shows >3% net edge after fees. Paper trade 200+ cycles and Brier score confirms calibration. You start live with $500 or less bankroll and $5 max per trade. Tax logging is built from day one. Do not pursue if expecting income before March 20, 2026 (Grand Rounds). This is a post-Grand Rounds project.

SECTION 6: EDGE-FINDING IN OTHER MARKETS — SYSTEMATIC GUIDE

6.1 The Universal Edge-Detection Framework

The question to ask about any Kalshi market: Is there an external data source that prices this event more accurately than Kalshi's current market price? If yes, and if the discrepancy exceeds fees plus execution costs, there is an exploitable edge.

6.2 BTC/Crypto Short-Interval Markets (Your Primary Target)

Data source: Binance spot price + Coinbase spot price (median of both). Mechanism: Kalshi's internal pricing update lags behind crypto exchange spot price movements by seconds to minutes near contract resolution. When spot price has already crossed the resolution threshold with high confidence, the corresponding Kalshi contract is still priced at near-50/50 even though the outcome is nearly certain. Kelly-sized bet on the near-certain side captures this lag.

Key questions to answer in backtest: How many seconds does the average lag last? What minimum price certainty threshold (distance from strike in sigma terms) predicts a Kalshi contract mispricing above 3 cents? Does this edge hold across all BTC volatility regimes or only in low-volatility periods?

- Data source for backtesting: Binance historical tick data (free download via Binance API), Kalshi historical market data (via Kalshi API /markets endpoint for closed markets)
- Tool for lag analysis: Calculate rolling 15-minute BTC volatility, correlate with frequency and magnitude of Kalshi contract mispricings

6.3 Weather / Temperature Markets

Data source: National Weather Service probabilistic forecasts (api.weather.gov), NOAA hourly observations. Mechanism: Kalshi weather markets like 'Will temperature exceed X in NYC tomorrow?' are frequently priced by retail traders using intuition rather than probabilistic forecast data. NWS provides machine-readable probability distributions that are more accurate than Kalshi prices on thinly traded weather markets. If NWS says 85% probability of temperature exceeding 72F but Kalshi market is at 70 cents, that is a 15-cent edge before fees.

Specific prompt for Claude Code edge detection: Build a script that pulls NWS hourly probabilistic forecast data for the target city and date, converts it to the implied probability of each Kalshi temperature contract resolving YES, and flags markets where |NWS_probability - Kalshi_price| > 0.05 after fee adjustment.

6.4 Economic Data Markets (CPI, Fed Rate, Unemployment)

Data source: Fed Funds futures (CME Group), Cleveland Fed Inflation Nowcast (free API), Atlanta Fed GDPNow. Mechanism: These markets have deeply liquid external pricing mechanisms with institutional money. Kalshi's retail-dominated orderbook frequently lags behind Fed Funds futures pricing by hours or days, especially when the FOMC meeting is still weeks away. When Fed Funds futures price a 78% probability of a rate cut and Kalshi's rate-cut market is at 65 cents, that is a 13-cent edge.

This is the most credible systematic edge on Kalshi for a non-HFT operator. The data sources are free, machine-readable, updated in near-real-time, and deeply researched by institutional actors who have priced in all public information. Kalshi retail prices frequently lag behind.

- Fed Funds futures: CME FedWatch Tool API (free)
- Inflation: Cleveland Fed Inflation Nowcast (free JSON endpoint)
- GDP: Atlanta Fed GDPNow (free, updated frequently during the quarter)
- Employment: Real-time labor market data from BLS (free)

6.5 Sports Markets — The Cautionary Category

Sports markets drove 89% of Kalshi's 2025 fee revenue. That means it is also where the most sophisticated prediction is happening and where the most retail money is losing. For a bot operator, sports markets have the highest volume (good for liquidity) but also the most competition from sharp bettors with proprietary models.

The edge category that remains viable in sports: late-breaking injury news. When a starting quarterback is ruled out 90 minutes before kickoff, Kalshi's markets take 3-10 minutes to fully reprice. A bot monitoring ESPN injury reports via RSS and NFL official injury designations via API can enter before the market corrects. This is essentially the same lag-exploitation mechanism as the BTC strategy but for sports.

Sports market edge questions for Claude Code: Build a bot that monitors official NFL/NBA injury report APIs and ESPN injury RSS feeds. When a player with >15% win probability impact is downgraded, calculate the implied new win probability using a basic Elo-adjusted model, and enter Kalshi markets where the price has not yet adjusted, as long as edge exceeds 3 cents after fees.

6.6 Correlated Market Logic Arbitrage

This is the most underexplored edge category for retail bots. Kalshi lists hundreds of markets on related events that should be mathematically constrained by each other. Example: If 'Trump wins 2028' is at 65 cents, then 'Republican wins 2028' must logically be at or above 65 cents (since Trump is a Republican). If 'Republican wins 2028' is at 60 cents, one of those two markets is mispriced relative to the other.

The bot needed: Continuously monitor 100+ active markets. Identify logical relationships (same-event different-outcomes, implied probabilities that must sum to 100%, candidate markets within the same election). When logical constraints are violated by more than 3 cents after fees, enter the mispriced side. Sweet spot: markets with greater than $100,000 liquidity where logical violations exceed 3%.

Real performance data from similar strategies: 70-80% win rate with average holding period of 2.3 days. This is chess, not roulette. You are exploiting market inefficiency, not predicting the future.

6.7 Edge Detection Prompt Template for Claude Code

Use this prompt structure when asking Claude Code to identify edges in any new market category:

```
For market category: [CATEGORY]
1. What external data source prices this event more accurately than retail intuition?
2. Is that data source free, machine-readable, and updated frequently enough to be actionable?
3. What is the expected magnitude of mispricing (in cents per contract)?
4. What is the expected frequency of mispricing events per day?
5. What is the expected liquidity depth (max contracts fillable before spread closes)?
6. What is the expected edge after Kalshi taker fee: ceil(0.07 x contracts x price x (1-price))?
7. What is the estimated time window before the market corrects itself?
8. Build a data pipeline to pull this external source. Build an edge detector that compares it to Kalshi prices. Simulate 30 days of paper trades.
```

SECTION 7: PLUGIN GUIDE — GSD AND SUPERPOWERS FOR THIS PROJECT

7.1 Superpowers — What It Does and How to Use It

Superpowers is a Claude Code plugin that accumulated 29,000+ GitHub stars and was officially accepted into Anthropic's Claude Code plugin marketplace on January 15, 2026. It is not developed by Anthropic but is a quality third-party tool. It is completely free, open-source, MIT license.

Superpowers transforms Claude Code from 'helpful assistant that sometimes goes off the rails' into 'systematic development partner that follows proven processes.' The structured workflow prevents the most common failure modes: skipped tests, missing edge cases, and features that drift from requirements.

Core features: Socratic Brainstorming (reflective design before code), Micro-Task Planning (breakdown into 2-5 minute tasks), Test-Driven Development (Red-Green-Refactor — tests must fail before implementation), Subagent-driven development with built-in code review, Systematic debugging with four-phase methodology (root cause investigation before any fixes), automatic Git commits after each task.

Superpowers Slash Commands — Use These for This Project

- /superpowers:brainstorm — Use before ANY new module. Claude will ask clarifying questions about requirements before writing code. Required before starting each of the 9 build phases.
- /superpowers:write-plan — Creates a detailed implementation plan after brainstorming. Use before starting Phase 2 (edge_detector.py) and Phase 4 (risk_manager.py) — the most complex modules.

- /superpowers:execute-plan — Runs the implementation plan in batches with review checkpoints. Prevents context drift across long coding sessions.

Specific Superpowers Usage Per Build Phase

Build Phase	Superpowers Command	What It Does For You
Phase 1: kalshi_client.py	/brainstorm then /execute-plan	Forces Claude to ask about auth flow, WebSocket vs REST, rate limit handling before writing auth code
Phase 2: edge_detector.py	/brainstorm then /write-plan then /execute-plan	This is the most complex module. TDD here is mandatory — edge detection logic must have failing tests first
Phase 3: logger.py	/brainstorm	SQLite schema design needs upfront planning — changing schema later breaks tax records
Phase 4: risk_manager.py	/brainstorm then /write-plan	Kill switch logic and Kelly sizing both need formal test coverage before any live exposure
Phase 5: calibration.py	/brainstorm	Brier score calculation must be mathematically verified before becoming the live-deploy gate
Phase 6: executor.py	/brainstorm then /execute-plan	Order placement has real-money consequences. TDD gates are critical here.
Phase 7: dashboard.py	/brainstorm	Streamlit is lower stakes but SQLite queries need to match logger.py schema exactly

Superpowers TDD Gate — Non-Negotiable

If Claude writes code before tests, the Superpowers skill instructs it to delete that code and start over. This is the iron rule. For this project, it means: every function in edge_detector.py must have a test that fails before the function is written. For example: test_edge_detected_when_btc_spot_diverges_from_kalshi_price() must fail before edge_detector() is implemented. test_no_edge_returned_when_spread_below_fee_threshold() must fail before the fee comparison logic is written.

7.2 GSD (Get-Shit-Done) — What It Does and How to Use It

GSD is a spec-driven development system built by TACHES, a solo developer who describes himself as someone who does not write code — Claude Code does. It was trusted by engineers at Amazon, Google, Shopify, and Webflow. It is available via npm: npx get-shit-done-cc --global to install globally to ~/.claude/

GSD is the context engineering layer that makes Claude Code reliable. It uses context engineering, XML prompt formatting, subagent orchestration, and state management behind the scenes. What you see is a few commands that just work. It is designed for solo developers building real things, not enterprise theater.

GSD addresses context degradation through fresh subagent contexts and atomic task planning. Each subagent receives a clean 200,000 token context window, ensuring Task 50 maintains the same quality as Task 1. The main Claude session remains at 30-40% context utilization while subagents perform heavy lifting. Every task generates an individual git commit, enabling granular rollback via git bisect.

GSD Slash Commands — Use These for This Project

- /gsd:new-project — Run this FIRST at session start after Grand Rounds. GSD extracts project requirements through a structured questioning workflow. Provide it this document as context.
- /gsd:create-roadmap — After new-project questioning is complete, creates the phased roadmap matching the 9-phase build plan in Section 8 of this document.
- /gsd:plan-phase — Use at the start of each build phase. Creates an atomic task plan for that phase with git-commitable units of work.
- /gsd:execute-plan — Executes the plan with subagents. Each subagent is fresh context — no accumulated confusion from long sessions.

- /gsd:verify — After each phase, verifies deliverables against the plan. Run this before moving to the next phase.
- /gsd:settings — Configure model profiles (Opus for planning, Sonnet for execution, Haiku for simple tasks). This controls token spend across phases.

GSD vs. Superpowers — When to Use Which

Use GSD for project-level orchestration: starting a new phase, creating the overall roadmap, managing multi-phase execution, and git commit hygiene. Use Superpowers for module-level discipline: brainstorming before each module, TDD enforcement within each module, debugging when something breaks, systematic code review.

They are complementary. The workflow is: /gsd:plan-phase to define atomic tasks for Phase N, then /superpowers:brainstorm before the first task, then Superpowers TDD through each task, then /gsd:verify at phase completion.

Context7 Plugin — Use for Kalshi API Accuracy

Context7 is the MCP server that pulls live, version-specific library documentation into Claude Code context. Always use it before writing any Kalshi API code. It prevents Claude Code from hallucinating deprecated endpoints or wrong parameter names.

How to invoke: When starting any session involving Kalshi API code, say 'Use Context7 to pull the current Kalshi API documentation for the /markets, /orders, and /portfolio endpoints before writing any API client code.' This ensures the WebSocket subscription syntax, authentication headers, and rate limit parameters are pulled from the actual live documentation, not Claude's training data.

GitHub Plugin — Use for Session Continuity

The GitHub plugin enables auto-commit after each completed task. Configure it to commit after every GSD task completion. The commit message should be structured as: [PHASE-N] [MODULE]: description of what was built/fixed. Example: [PHASE-2] edge_detector: add fee-adjusted minimum spread threshold with unit test. This creates a git history that functions as a session handoff log — any new Claude Code session can read the commit log to understand where the project is.

SECTION 8: COMPLETE BUILD ROADMAP — START TO FINISH

TIMELINE NOTE:
Start after Grand Rounds approximately March 20, 2026. Expected completion of paper-trading gate: 6-8 weeks. Expected first live capital deployment: May 2026. Budget: approximately 2 hours per session, 3-4 sessions per week.

Pre-Build: Environment Setup (Day 1 — ~2 Hours)

Before touching Claude Code, complete all of the following:

1. Open Kalshi account at kalshi.com — complete full KYC verification (allow up to 2 weeks based on reported delays)
2. Navigate to Settings → API → generate RSA key pair → save kalshi_private_key file in a secure location NOT in the git repo
3. Create a private GitHub repository named kalshi-btc-bot
4. Install locally: Python 3.11+, run: pip install kalshi-python websockets pandas requests python-dotenv
5. Create .env file in repo root with: KALSHI_API_KEY_ID=[your key], KALSHI_PRIVATE_KEY_PATH=./kalshi_private_key, DRY_RUN=true,

BINANCE_BASE_URL=https://api.binance.com, COINBASE_BASE_URL=https://api.coinbase.com, DB_PATH=./trades.db, MAX_TRADE_USD=5.00, DAILY_LOSS_LIMIT=25.00

6. Create .gitignore containing: .env, kalshi_private_key, *.db
7. Install GSD globally: npx get-shit-done-cc --global
8. Verify Superpowers is installed (already confirmed in your setup)
9. Create CLAUDE.md in repo root — paste the contents from Section 9 of this document

Phase 1: Kalshi API Client and BTC Price Feed (Session 1)

Invoke: /superpowers:brainstorm first. Then ask Claude Code to build:

kalshi_client.py — Authenticates with Kalshi API using RSA key from .env, fetches BTC 15-minute markets, subscribes to WebSocket orderbook feed, returns real-time YES/NO prices. Handles rate limits with exponential backoff (2s, 4s, 8s). Logs all API errors to file. Raises custom KalshiRateLimitError and KalshiFeedError exceptions. Never uses market orders.

price_feed.py — Pulls live BTC spot price from Binance AND Coinbase public REST APIs simultaneously, returns the median of both, polls every 5 seconds, raises PriceFeedError if feeds diverge by more than $100 or if either feed fails for more than 30 seconds.

Test both independently before proceeding: python kalshi_client.py (should print current BTC market YES/NO prices), python price_feed.py (should print median BTC price every 5 seconds). Do not move to Phase 2 until both work independently.

Phase 2: Edge Detection Engine (Session 2)

Invoke: /superpowers:brainstorm then /superpowers:write-plan. This is the mathematical core. TDD is mandatory.

edge_detector.py — Given BTC spot price from price_feed.py and current Kalshi market YES price from kalshi_client.py, calculate implied probability from BTC 15m market contract parameters. Calculate edge = |implied_prob - kalshi_price|. Apply Kalshi taker fee formula: fee = ceil(0.07 x contracts x price x (1 - price)). Return a trade signal only if edge > 0.04 (4 cents) after subtracting estimated fee. If no edge survives fees, return None. Include confidence_interval calculation based on current BTC volatility (higher volatility = wider interval = smaller effective edge).

Tests that must fail before implementation: test_no_signal_when_edge_below_fee_threshold(), test_signal_returned_when_btc_clearly_above_strike(), test_fee_formula_matches_kalshi_official_formula(), test_none_returned_on_feed_error().

Phase 3: SQLite Tax Logger (Session 2, continued)

logger.py — SQLite database with complete trades table (all fields from Section 4.4 of this document). Every trade writes a complete row — no partial rows allowed. Build get_daily_summary() returning gross profit, fees, net profit, and running tax_basis for the current calendar year. Build get_brier_inputs() returning all rows where outcome is not null for Brier score calculation. Build export_tax_csv() that exports all resolved trades to CSV with proper column headers for tax reporting.

Phase 4: Risk Management and Kill Switch (Session 3)

Invoke: /superpowers:brainstorm then /superpowers:write-plan.

risk_manager.py — Enforces: max $5 per trade enforced by contract count x price, daily loss limit $25 auto-halt if hit, daily loss resets at midnight UTC, max 3 open positions simultaneously, Kelly Criterion position sizing using current Brier score from calibration.py (if Brier score > 0.20, reduce sizing by 50%). Build kill_switch() that cancels all open orders via Kalshi API and halts the bot. Kill switch triggers on: daily loss limit breach, API error rate >10% in 5 minutes, price feed failure >30 seconds, manual flag in .env (KILL=true). Log every kill switch trigger to a separate kill_log table in SQLite.

Phase 5: Brier Score Calibration Tracker (Session 3, continued)

calibration.py — After each market resolves, retrieve outcome from Kalshi API, compare to the probability estimate made at trade entry (stored in logger.py). Calculate running Brier score = mean((forecast - outcome)^2) across all resolved markets. Brier score of 0.25 equals coin flip — no edge. Bot must be hard-blocked from live trading (DRY_RUN forced to true) until Brier score drops below 0.20 across at least 100 resolved markets. Display current Brier score in dashboard. Warn in logs when approaching 100-market threshold so you can evaluate readiness.

Phase 6: Order Execution Engine (Session 4)

Invoke: /superpowers:brainstorm then /superpowers:execute-plan.

executor.py — When edge_detector returns a signal and risk_manager approves, place a LIMIT order via Kalshi API at the detected edge price. If DRY_RUN=true, simulate fill and log to SQLite without placing real order (simulate fill as immediate at entry price for paper trading purposes). If DRY_RUN=false, place real limit order. Log all fills. Check fill status every 10 seconds. Cancel unfilled orders after 60 seconds. Never use market orders under any circumstances. On failed order placement, log error and skip (do not retry more than once). On kill_switch trigger, cancel all open orders before halting.

Phase 7: Streamlit Dashboard (Session 5)

dashboard.py — Streamlit application showing: live PnL today (gross and net after fees), running tax basis year-to-date (ordinary income), Brier score with current market count, last 20 trades table (timestamp, side, entry, exit, net_profit, outcome), kill switch toggle button, DRY_RUN status indicator, daily loss limit remaining. Pull all data from SQLite. Auto-refresh every 30 seconds. Show red banner if daily loss limit is within $5 of trigger. Show orange banner if Brier score is above 0.20 (live trading disabled). Show green banner if all systems nominal.

Phase 8: Paper Trading Gate (Sessions 6-8 — 2+ Weeks of Running)

Run DRY_RUN=true continuously. Do not touch DRY_RUN=false until all of the following are simultaneously true: 200+ resolved markets logged in SQLite, Brier score below 0.20, net simulated profit after fees is positive, no kill switch triggers in the last 48 hours of continuous operation, dashboard is accessible and displaying correct data.

During paper trading, log everything even if nothing is traded. If the market is live and you are monitoring it, record that you monitored it. Track: how many 15-minute BTC intervals passed with zero signal (expected to be the majority), how many intervals had a signal but edge did not survive fees (important data), how many intervals had a tradeable signal. This data determines whether the strategy is fundamentally sound before live capital is at risk.

Phase 9: Live Deployment (Session 9+)

Set DRY_RUN=false only after the Phase 8 gate is cleared. Start with $200 bankroll. $5 max trade enforced. Monitor the dashboard daily. After 30 live trading days, evaluate: Is net PnL positive after fees? Is Brier score holding below 0.20 as new markets resolve? Are there patterns in which intervals produce edges and which do not? Only after confirming sustained positive net PnL over 30 live days should bankroll be increased, and only in $100 increments.

Evaluate CLV — are your entry prices better than closing prices? If your entry prices are consistently worse than closing prices (negative CLV), the edge is noise or you are consistently entering too late in the lag window.

SECTION 9: CLAUDE.md TEMPLATE — PASTE THIS INTO REPO ROOT

Copy the following exactly into CLAUDE.md in the kalshi-btc-bot repository root. Update the phase and last session notes at each session start.

```
# KALSHI BOT — CLAUDE.md
## Project
Kalshi BTC 15-minute prediction market bot. BTC price lag strategy vs.
Binance/Coinbase spot feeds.
## Hard Constraints — Never Violate
- DRY_RUN=true until 200+ paper trades, Brier score <0.20
- $5 maximum per trade — programmatic enforcement only
- $25 daily loss kill switch — auto-halt, midnight UTC reset
- Limit orders ONLY. No market orders. Ever.
- Kelly Criterion sizing — no gut-feel sizing
- Log every trade to SQLite with all 14 fields from Section 4.4 of reference doc
- Do not remove kill switch logic under any circumstances
## Current Phase
[UPDATE EACH SESSION: Phase N — ModuleName]
## Last Session Output
[UPDATE EACH SESSION: filename.py — what was built/last status]
## Stack
Python 3.11 | Kalshi REST + WebSocket API | SQLite | Streamlit
## Fee Formula
fee = ceil(0.07 x contracts x price x (1 - price))
## Plugins Active
Superpowers (/brainstorm before every module, TDD mandatory)
GSD (/gsd:plan-phase at phase start, /gsd:verify at phase end)
Context7 (pull Kalshi API docs before any API client code)
GitHub (auto-commit after each GSD task)
## Reference Document
KALSHI_BOT_COMPLETE_REFERENCE.docx — read this at session start
```

SECTION 10: CRITICAL QUESTIONS RAISED IN RESEARCH — FOR ONGOING REFERENCE

These are the questions explicitly asked and answered in the research conversation. They represent the key decision points that must not be forgotten as the project progresses.

Platform and Market Selection

- Q: Is Kalshi the right platform or am I already handicapping myself before writing a line of code? A: Kalshi is correct for now because Polymarket US is still invite-only, but build platform-agnostic code from day one.
- Q: Should I pursue this Kalshi bot? A: Yes, conditionally — only if backtest shows >3% net edge after fees, paper trading validates Brier score <0.20, and you treat it as supplemental income testing rather than primary income generation.
- Q: What are platform risks nobody talks about? A: Resolution disputes (Kalshi changed rules on live markets mid-trading), withdrawal failures, state-level bans in at least 4 states, insider trading enforcement creating legal exposure.

Mathematical Edge

- Q: Have you done the fee math per-trade, not just in theory? A: The fee formula is ceil(0.07 x contracts x price x (1-price)). At $5/trade and 50-cent contract: approximately 2 cents per contract. Must be calculated explicitly per trade — never estimated.
- Q: What is the BTC lag strategy's actual verified edge mechanism? A: Options markets imply BTC probabilities more accurately than Kalshi's retail-dominated orderbook, especially near contract

resolution. Gap of 5-15 cents has been documented. Post-fee validation on Kalshi specifically has not been peer-reviewed.

- Q: What does 'calibration' actually mean? A: When you say 60% probability, the event happens 60% of the time. Measured by Brier score (lower is better). 0.25 = coin flip. Target: below 0.20.

Infrastructure and Competition

- Q: What competition am I walking into? A: 73% of arbitrage profits captured by sub-100ms bots. Pure arb is dead for retail. Viable categories: market making, correlated market logic, and event-driven strategies where speed is less critical than model accuracy.
- Q: Can I actually scale to $50/day? A: Mathematically achievable based on documented $16.80/trade averages at 3 trades/day. Practically unvalidated for your specific strategy on Kalshi BTC 15-minute markets. Paper trade first.
- Q: What is the one thing nobody asked that remains crucial? A: Tax treatment. Kalshi issues 1099s. IRS guidance is unsettled. Three competing treatments exist. Most defensible for your situation: ordinary income on Schedule 1.

Build Quality

- Q: What does the Claude Code build actually need to not be a paper exercise? A: Brier score tracker, fee-adjusted EV per trade, Kelly Criterion sizing, WebSocket feed (not REST polling), kill switch tied to daily drawdown, complete SQLite audit trail for tax purposes.
- Q: Which plugins and skills help this project specifically? A: GSD for project-level orchestration and session continuity, Superpowers for module-level TDD discipline, Context7 for Kalshi API accuracy, GitHub for commit-based session handoff.

Tax and Ongoing Compliance

- Q: How does tax factor into the profitability math? A: At 24% ordinary income rate, the $50/day gross target yields approximately $31-37/day after taxes. Quarterly estimated payments required above $1,000 annual profit. Reconstruct every trade from SQLite — do not rely on Kalshi's platform summary.
- Q: Is Section 1256 treatment applicable? A: Contested. One school says yes because Kalshi is a CFTC-designated contract market. Another school says no because event-based binary contracts do not clearly fit the statutory definition. Conservative approach: ordinary income until a qualified prediction-market CPA reviews your situation.

SECTION 11: KEY RESEARCH SOURCES REFERENCED

- GWU/UCD Academic Working Papers on Kalshi favorite-longshot bias and pricing accuracy (2025/2026)
- Bocconi Students Investment Club: 'Well, CAN WE PREDICT?' — backtesting GBM models on Kalshi BTC contracts
- CoinDesk (February 2026): 'How AI is helping retail traders exploit prediction market glitches'
- Medium/Illumination (February 2026): 'Beyond Simple Arbitrage: 4 Polymarket Strategies Bots Actually Profit From in 2026'
- Sacra Research: Kalshi revenue, valuation and funding (2025)
- DeFiRate.com: Prediction market fees comprehensive comparison (January 2026)
- Camuso CPA: Kalshi Tax Reporting and Section 1256 analysis (February 2026)
- Monaco CPA: Prediction Market Taxes 2025 Returns (March 2026)
- CNBC: 'Prediction markets are booming. How to tax winnings is anyone's guess' (December 2025)
- Bh-Ayush/Kalshi-CryptoBot GitHub — reference implementation for BTC 15-minute strategy
- Jacob Ferraiolo TSA Trading Bot Blog Series — practitioner documentation with honest loss accounting
- BuiltThisWeek transcript: 'We Built an AI Trading Bot for Prediction Markets'

- QuantVPS: Cross-Market Arbitrage technical guide
- NavnoorBawa Substack: Building a Prediction Market Arbitrage Bot — technical implementation
- Superpowers plugin: github.com/obra/superpowers — official documentation
- GSD plugin: github.com/gsd-build/get-shit-done — official documentation and installation guide
- Kalshi official API documentation: trading-api.readme.kalshi.com
- Kalshi official fee schedule: kalshi.com/fee-schedule
- Kalshi official tax info: help.kalshi.com/documents-and-taxes/tax-info
- DeFiRate.com: Kalshi vs. Polymarket comprehensive comparison (March 2026)

END OF REFERENCE DOCUMENT